“Compiler Design

Unit 3
Parser

Available on notes.arpanadhikari7.com.np

Introduction

- Parsing is the second stage of the compiler's front end.
- The parser derives a syntactic structure for the program, fitting the words into a grammatical model of the source programming language.
- If the parser determines that the input stream is a valid program, it builds a concrete model of the program for use by the later phases of compilation.
- If the input stream is not a valid program, the parser reports the problem and appropriate diagnostic information to the user.

Available on notes.arpanadhikari7.com.np

- A compiler's parser has the primary responsibility for recognizing syntax—that is, for determining if the program being compiled is a valid sentence in the syntactic model of the programming language.

- That model is expressed as a formal grammar G; if some string of words s is in the language defined by G we say that G derives s.

- For a stream of words s and a grammar G, the parser tries to build a constructive proof that s can be derived in G -> a **process called parsing.**

Available on notes.arpanadhikari7.com.np

- A string of words s (composed of terminals from Σ) is said to be **derived by G** (or *s* is in the language L(G)) if, starting from S, we can apply a sequence of production rules from P to generate s.
- The language L(G) is the set of all strings (sentences) that can be derived from *G*.
- Let's define a simple grammar *G* for a tiny fragment of English:
- N={S,NP,VP,Det,N,V}
- Σ={the,cat,dog,chased,slept}
- P=
  - S→NP VP
  - NP→Det N
  - VP→V NP | V
  - Det→the
  - N→cat | dog
  - V→chased | slept
- Start symbol: S derive **"the cat chased the dog":**
- If this derivation succeeds, the string "the cat chased the dog" is in L(G)*L*(*G*).

Available on notes.arpanadhikari7.com.np

Role of The Parser

- Parser for any grammar is program that takes as input string w and produces as output either a parse tree for w , if w is a valid sentences of grammar or error message indicating that w is not a valid sentences of given grammar.

- The goal of the parser is to determine the syntactic validity of a source string is valid, a tree is built for use by the subsequent phases of the computer.

- The tree reflects the sequence of derivations or reduction used during the parser. Hence, it is called parse tree.

Available on notes.arpanadhikari7.com.np

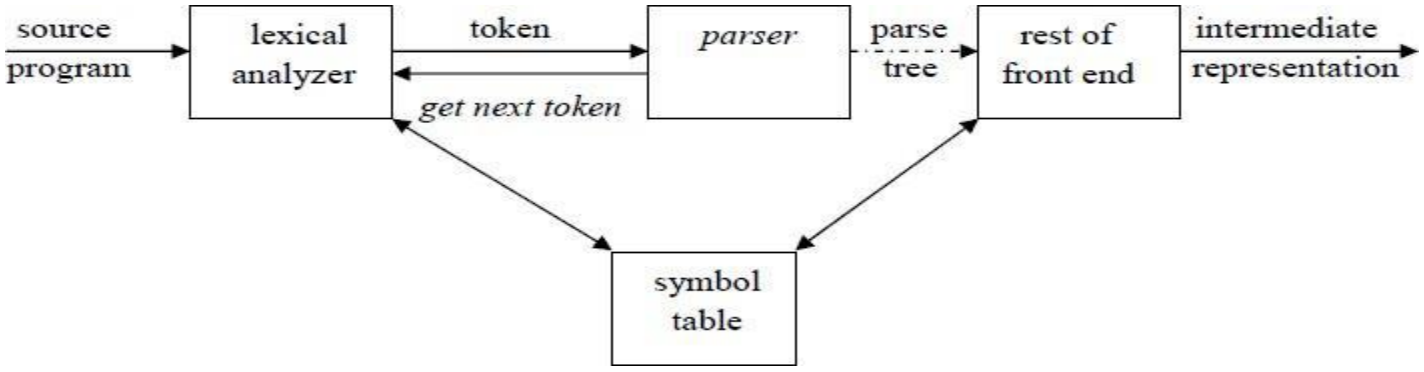
- If string is invalid, the parse has to issue diagnostic message identifying the nature and cause of the errors in string.
- Every elementary subtree in the parse tree corresponds to a production of the grammar.

Available on notes.arpanadhikari7.com.np

The two types of parsers employed are:

a. Top down parser: which build parse trees from top(root) to bottom(leaves)

b. Bottom up parser: which build parse trees from leaves and work up the root.



Available on notes.arpanadhikari7.com.np

Context-Free Grammars

- A Context-Free Grammar is a fundamental concept in compiler design used to describe the syntax of programming languages.
- It's a formal grammar that defines how to generate all possible strings in a given formal language.
- Context-free grammars can generate context-free languages.
- .

Available on notes.arpanadhikari7.com.np

- To describe programming language syntax, we need a more powerful notation.
- The Context-free grammar traditional solution is to use a context-free grammar (CGF).
- A context-free grammar, G, is a set of rules that describe how to form sentences.
- The collection of sentences that can be derived from G is called the language defined by G, denoted G.
- The set of languages defined by context free grammars is called the set of context-free languages.

Available on notes.arpanadhikari7.com.np

- A context-free grammar (grammar for short) consists of terminals, nonterminals, a start symbol, and productions

- **A set of terminal symbols** which are the characters that appear in the language/strings generated by the grammar. Terminal symbols never appear on the left-hand side of the production rule and are always on the right-hand side.

- **A set of non-terminal symbols (or variables)** which are placeholders for patterns of terminal symbols that can be generated by the non-terminal symbols. These are the symbols that will always appear on the left-hand side of the production rules, though they can be included on the right-hand side.

Available on notes.arpanadhikari7.com.np

- A set of production rules which are the rules for replacing non-terminal symbols. Production rules have the following form: variable → string of variables and terminals.
- A start symbol which is a special non-terminal symbol that appears in the initial string generated by the grammar.
- Mathematically, CFG can be defined as 4 – tuple (V, T, P, S), where,

V: finite set of Variables or Non-terminals that are used to define the grammar denoting the combination of terminal or no-terminals or both

T: Terminals, the basic symbols of the sentences. Terminals are indivisible units

P: Production rule that defines the combination of terminals or non-terminals or both for particular non-terminal

S: It is the special non-terminal symbol called start symbol

Available on notes.arpanadhikari7.com.np

- The construction of a parse tree can be made precise by taking a derivational view, in which productions are treated as rewriting rules.
- Beginning with the start symbol, each rewriting step replaces a non-terminal by the body of one of its productions.
- This derivational view corresponds to the top-down construction of a parse tree, but the precision afforded by derivations will be especially helpful when bottom-up parsing.

Available on notes.arpanadhikari7.com.np

- At each point in this derivation process, the string is a collection of terminal or non terminal symbols.
- Such a string is called a *sentential form if it occurs in some step of* a valid derivation in some step of a valid derivation.
- A **sentential form** is any string of terminals and non-terminals that can be derived from the start symbol of a context-free grammar (CFG) in one or more steps.
- Any sentential form can be derived from the start symbol in zero or more steps.
- Similarly, from any sentential form we can derive a valid sentence in zero or more steps.

Available on notes.arpanadhikari7.com.np

- During parsing, we take two decisions for some sentential form of input:
  - Deciding the non-terminal which is to be replaced.
  - Deciding the production rule, by which, the non-terminal will be replaced.

- To decide which non-terminal to be replaced with production rule, we can have two options.

Left-most derivation

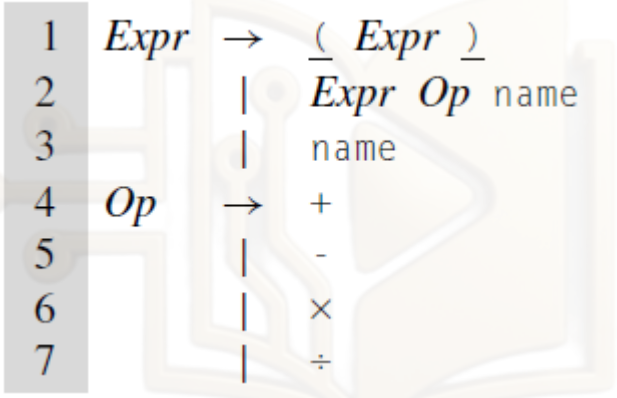
- If the sentential form of an input is scanned and replaced from left to right, it is called left-most derivation.

Right-most derivation

- If we scan and replace the input with production rules, from right to left, it is known as right-most derivation

Available on notes.arpanadhikari7.com.np

- Consider the language of expressions with parentheses.



1	*Expr*	→	(*Expr*)
2		|	*Expr* *Op* name
3		|	name
4	*Op*	→	+
5		|	-
6		|	×
7		|	÷

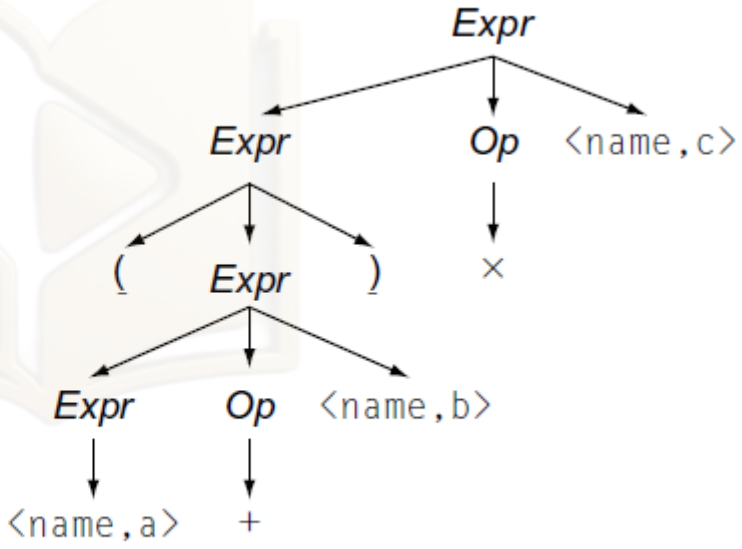
- Beginning with the start symbol, Expr, we can generate two kinds of subterms: parenthesized subterms, with rule 1, or plain subterms, with rule 2.

- To generate the sentence "(a + b) × c", we can use the following rewrite sequence (2,6,1,2,4,3),

Available on notes.arpanadhikari7.com.np

Rule	Sentential Form
	Expr
2	*Expr Op* name
6	*Expr* × name
1	(*Expr*) × name
2	(*Expr Op* name) × name
4	(*Expr* + name) × name
3	(name + name) × name

Rightmost Derivation of (a + b) × c

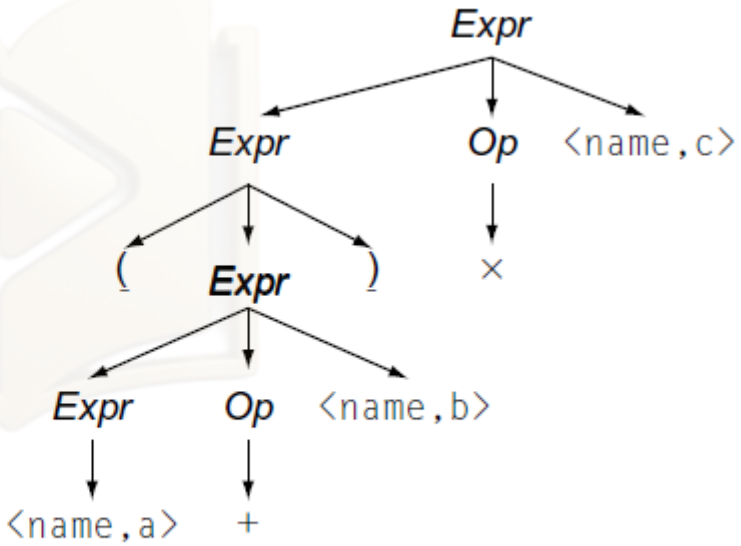


Corresponding Parse Tree

Available on notes.arpanadhikari7.com.np

Rule	Sentential Form
	Expr
2	*Expr Op* name
1	(*Expr*) *Op* name
2	(*Expr Op* name) *Op* name
3	(name *Op* name) *Op* name
4	(name + name) *Op* name
6	(name + name) × name

Leftmost Derivation of (a + b) x c



Corresponding Parse Tree

Available on notes.arpanadhikari7.com.np

- The leftmost and rightmost derivations use the same set of rules; they apply those rules in a different order.
- Because a parse tree represents the rules applied, but not the order of their application, the parse trees for the two derivations are identical.
- From the compiler's perspective, it is important that each sentence in the language defined by a CGF has a unique rightmost (or leftmost) derivation.

Available on notes.arpanadhikari7.com.np

- If multiple rightmost (or leftmost) derivations exist for some sentence, then, at some point in the derivation, multiple distinct rewrites of the rightmost (or leftmost) nonterminal lead to the same sentence.

- A grammar in which multiple rightmost (or leftmost) derivations exist for a sentence is called a ambiguous grammar.

- An ambiguous grammar can produce multiple derivations and multiple parse trees.

Available on notes.arpanadhikari7.com.np

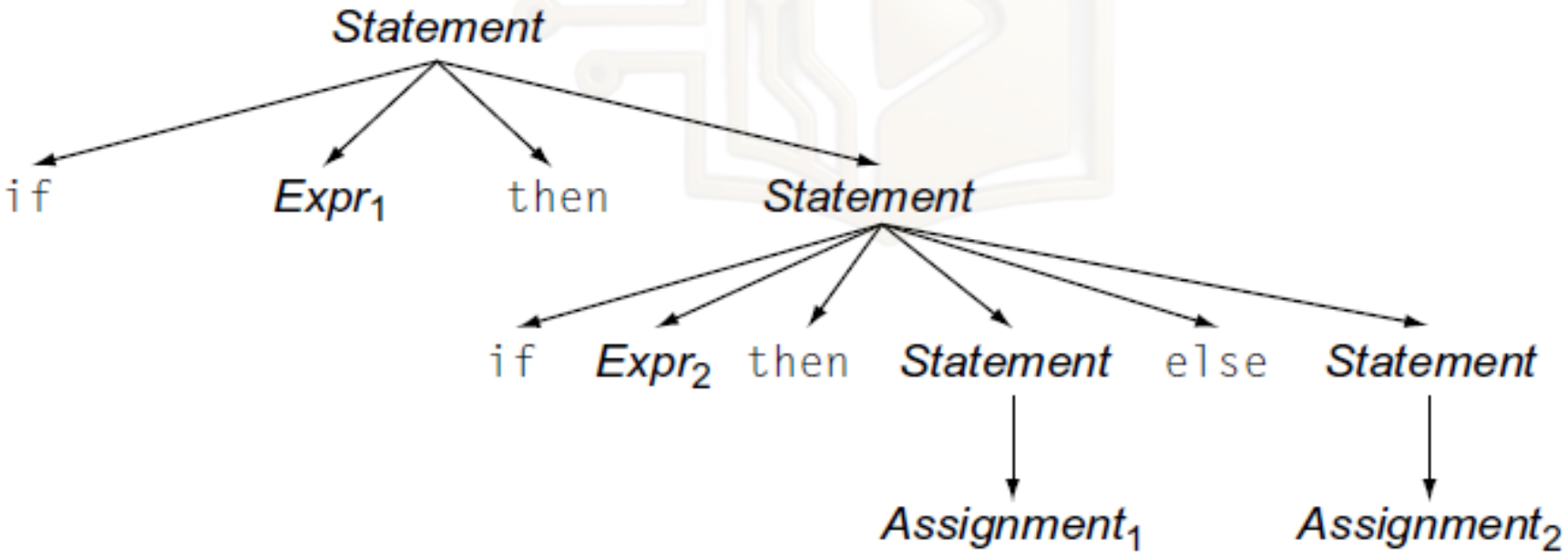
- The classic example of an ambiguous construct in the grammar for a programming language is the if-then-else construct of many Algol-like languages.
- The straightforward grammar for if-then-else might be

1	*Statement*	→	`if` *Expr* `then` *Statement* `else` *Statement*
2		|	`if` *Expr* `then` *Statement*
3		|	*Assignment*
4		|	*. . . other statements . . .*

- This fragment shows that the else is optional. Unfortunately, the code fragment if Expr1 then if Expr2 then Assignment1 else Assignment2 has two distinct rightmost derivations

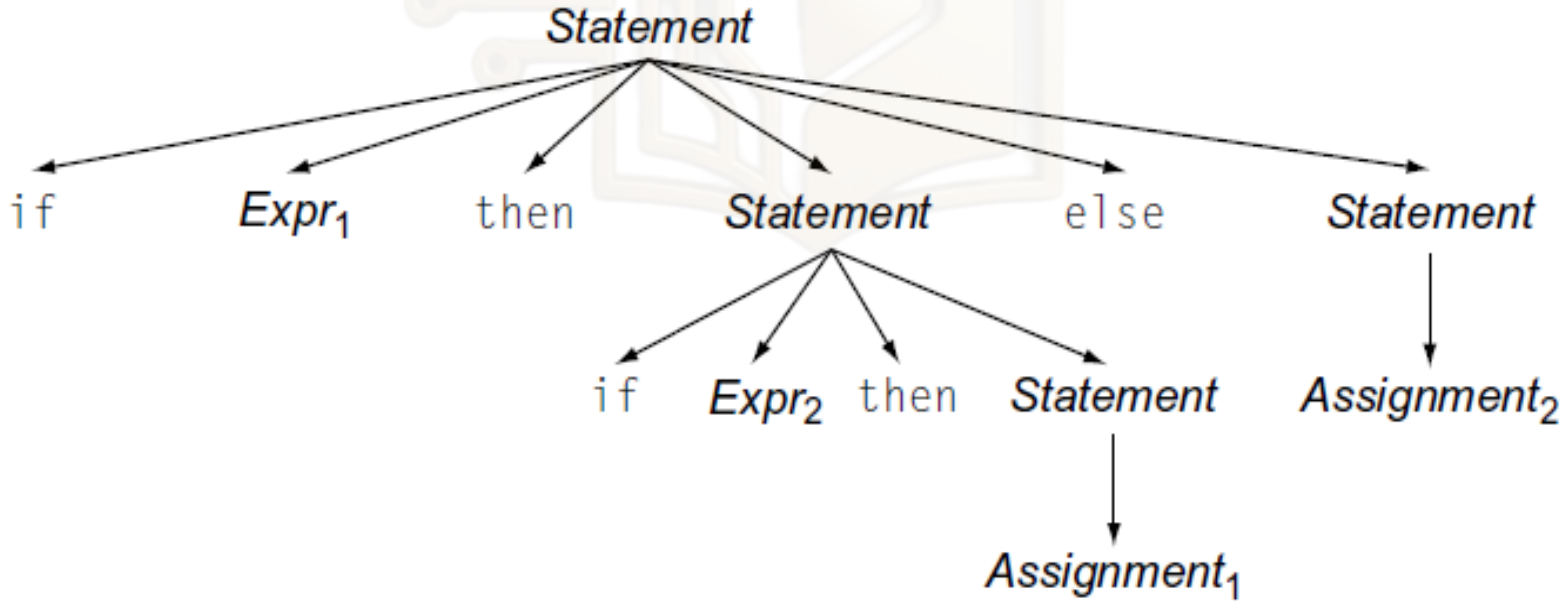
Available on notes.arpanadhikari7.com.np

- The difference between them is simple. The first derivation has Assignment2 controlled by the inner if, so Assignment2 executes when Expr1 is true and Expr2 is false:



Available on notes.arpanadhikari7.com.np

- The second derivation associates the else clause with the first if, so that Assignment2 executes when Expr1 is false, independent of the value of Expr2: else



Available on notes.arpanadhikari7.com.np

- To remove this ambiguity, the grammar must be modified to encode a rule that determines which if controls an else.
- To fix the if-then-else grammar, we can rewrite it as

1	*Statement*	→	`if` *Expr* `then` *Statement*
2		|	`if` *Expr* `then` *WithElse* `else` *Statement*
3		|	*Assignment*
4	*WithElse*	→	`if` *Expr* `then` *WithElse* `else` *WithElse*
5		|	*Assignment*

Available on notes.arpanadhikari7.com.np

- The solution restricts the set of statements that can occur in the then part of an if-then-else construct.
- It accepts the same set of sentences as the original grammar, but ensures that each else has an unambiguous match to a specific if.
- It encodes into the grammar a simple rule—bind each else to the innermost unclosed if.
- It has only one rightmost derivation for the example.

Available on notes.arpanadhikari7.com.np

Rule	Sentential Form
	Statement
1	`if` *Expr* `then` *Statement*
2	`if` *Expr* `then` `if` *Expr* `then` *WithElse* `else` *Statement*
3	`if` *Expr* `then` `if` *Expr* `then` *WithElse* `else` *Assignment*
5	`if` *Expr* `then` `if` *Expr* `then` *Assignment* `else` *Assignment*

Available on notes.arpanadhikari7.com.np

Top-down Parsing

- A top-down parser begins with the root of the parse tree and systematically extends the tree downward until its leaves match the classified words returned by the scanner.
- At each point, the process considers a partially built parse tree.
- It selects a non terminal symbol on the lower fringe of the tree and extends it by adding children that correspond to the right-hand side of some production for that non terminal

Available on notes.arpanadhikari7.com.np

- It cannot extend the frontier from a terminal.
- This process continues until either

a. **the fringe of the parse tree contains only terminal symbols, and the input** stream has been exhausted, or

b. **a clear mismatch occurs between the fringe of the partially built parse** tree and the input stream.

Available on notes.arpanadhikari7.com.np

Transforming a Grammar for Top-Down Parsing

- Transforming a grammar to make it suitable for top-down parsing (especially for **LL(1)** parsing) involves eliminating certain grammar features that cause non-determinism in parsing decisions.

- Here are the key steps to transform a grammar for top-down parsing:

Available on notes.arpanadhikari7.com.np

Eliminate Left Recursion

- A production of grammar is said to have **left recursion** if the leftmost variable of its RHS is same as variable of its LHS.
- A grammar containing a production having left recursion is called as Left Recursive Grammar.
- Top-down parsing methods cannot handle left recursion grammars, so a transformation that eliminates left recursion is needed.

Available on notes.arpanadhikari7.com.np

- If we have the left-recursive pair of productions-
- **$A \rightarrow A\alpha / \beta$** where β does not begin with an A.
- Then, we can eliminate left recursion by replacing the pair of productions with-

  $A \rightarrow \beta A'$

  $A' \rightarrow \alpha A' / \in$
- No matter how many A productions there are, we can eliminate immediate left recursion from them by the following technique. First we group the A production as
- $A \rightarrow A\alpha_1 | A\alpha_2 | \ldots | A\alpha_n | \beta_1 | \beta_2 | \ldots \beta_n$ where no βi begins with an A. then, we replace the A -productions by

$A \rightarrow \beta_1 A' | \beta_2 A' | \ldots | \beta_n A'$

$A' \rightarrow \alpha_1 A' | \alpha_2 A' | \ldots | \alpha_n A' | \in$

Available on notes.arpanadhikari7.com.np

Example of left recursion

1. $A \rightarrow ABd \mid Aa \mid a \quad B \rightarrow Be \mid b$

2. $S \rightarrow A \quad A \rightarrow Ad \mid Ae \mid aB \mid ac \; B \rightarrow bBc \mid f$

3. $S \rightarrow Aa \mid b \quad A \rightarrow Ac \mid Sd \mid \in$

4. $E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F \quad F \rightarrow (E) \mid id$

Available on notes.arpanadhikari7.com.np

Non Immediate Left Recursion

- This occurs in a chain like:

$$A \rightarrow B\alpha$$
$$B \rightarrow A\beta$$

To eliminate it, substitute productions until immediate left recursion appears, then apply the above method.

Available on notes.arpanadhikari7.com.np

Example

1. $S \rightarrow Aa \mid b \quad A \rightarrow Ac \mid Sd \mid \varepsilon$

2. $A \rightarrow B\,a \mid c \quad B \rightarrow A\,b \mid d$

3. $S \rightarrow Aa \mid b \quad A \rightarrow Sc \mid d$

Available on notes.arpanadhikari7.com.np

- Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive or top-down parsing.
- A grammar in which more than one production has common prefix is to be rewritten by factoring out the prefixes.
- Left factoring helps when two or more productions share a common prefix, making it hard to decide which production to use.
- The following algorithm is used to perform left factoring in the grammar-
- Suppose the grammar is in the form:
- $A \Rightarrow \alpha\beta1 \mid \alpha\beta2 \mid \alpha\beta3 \mid \ldots\ldots \mid \alpha\beta n \mid \gamma$.
- Where **A** is a non-terminal and **α** is the common prefix.

Available on notes.arpanadhikari7.com.np

- We will separate those productions with a common prefix and then add a new production rule in which the new non-terminal we introduced will derive those productions with a common prefix.

- $A \Rightarrow \alpha A' \mid \gamma$
- $A' \Rightarrow \beta 1 \mid \beta 2 \mid \beta 3 \mid \ldots\ldots \mid \beta n$

Available on notes.arpanadhikari7.com.np

Examples of left factoring

1. $A \rightarrow aB \mid aC \mid aD \quad B \rightarrow b \quad C \rightarrow c \quad D \rightarrow d$
2. $S \rightarrow iEtS \ / \ iEtSeS \ / \ a \quad E \rightarrow b$
3. $S \rightarrow bSSaaS \ / \ bSSaSb \ / \ bSb \ / \ a$

Available on notes.arpanadhikari7.com.np

Ensure the Grammar is LL(1)

- A grammar is **LL(1)** if for every non-terminal with multiple productions, the **FIRST sets** of the right-hand sides are disjoint, and if a production derives ϵ, then **FIRST** and **FOLLOW** sets must not overlap.

- **FIRST(X)**: Set of terminals that can begin strings derived from *X*.

- **FOLLOW(X)**: Set of terminals that can appear immediately after X in some derivation.

Available on notes.arpanadhikari7.com.np

Rules for first( ):

1. If X is terminal, then FIRST(X) is {X}.
2. If X → ε is a production, then add ε to FIRST(X).
3. If X is non-terminal and X → aα is a production then add a to FIRST(X).
4. If X is non-terminal and X → Y1 Y2…Yk is a production, then place a in FIRST(X) if for some i, a is in FIRST(Yi), and ε is in all of FIRST(Y1),…,FIRST(Yi-1); that is, Y1,….Yi-1 => ε. If ε is in FIRST(Yj) for all j=1,2,..,k, then add ε to FIRST(X).

Available on notes.arpanadhikari7.com.np

Rules for follow( ):

1. If S is a start symbol, then FOLLOW(S) contains $.

2. If there is a production $A \rightarrow \alpha B \beta$, then everything in FIRST(β) except ε is placed in follow(B).

3. If there is a production $A \rightarrow \alpha B$, or a production $A \rightarrow \alpha B \beta$ where FIRST(β) contains ε, then everything in FOLLOW(A) is in FOLLOW(B).

Available on notes.arpanadhikari7.com.np

Recursive-Descent Parsers

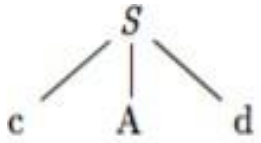
- Recursive descent parsing is one of the top-down parsing techniques that uses a set of recursive procedures to scan its input.
- This parsing method may involve backtracking, that is, making repeated scans of the input.

Example : Consider the grammar

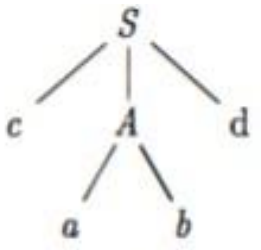
S→cAd

A→ ab | a

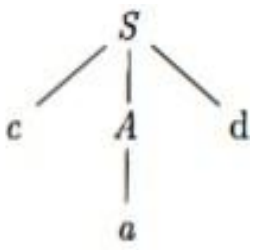
To construct a parse tree top-down for the input string w = cad



(a)



(b)



(c)

Available on notes.arpanadhikari7.com.np

Example for recursive decent parsing:

- A left-recursive grammar can cause a recursive-descent parser to go into an infinite loop.
- Hence, **elimination of left-recursion must be done before parsing.**
- Consider the grammar for arithmetic expressions

  $E \rightarrow E+T \mid T$

  $T \rightarrow T*F \mid F$

  $F \rightarrow (E) \mid id$
- After eliminating the left-recursion the grammar becomes,

  $E \rightarrow TE'$

  $E' \rightarrow +TE' \mid \varepsilon \quad T \rightarrow FT'$

  $T' \rightarrow *FT' \mid \varepsilon \quad F \rightarrow (E) \mid id$

Parse for the word w=id+id*id

Available on notes.arpanadhikari7.com.np

Table-Driven LL(1) Parsers(Non Recursive)

- Recursive descent parsers have many attractive features.
- They are actual pieces of code that can be read by programmers and extended.
- This makes it fairly easy to understand how parsing is done.
- Parsing procedures are also convenient places to add code to build ASTs, or to do type checking, or to generate code.
- A major drawback of recursive descent is that it is quite inconvenient to change the grammar being parsed.
- Any change, even a minor one, may force parsing procedures to be reprogrammed, as productions and predict sets are modified

Available on notes.arpanadhikari7.com.np

- To a less extent, recursive descent parsing is less efficient than it might be, since subprograms are called just to match a single token or to recognize a right hand side.
- An alternative to parsing procedures is to encode all prediction in a parsing table.
- A pre-programmed driver program can use a parse table (and list of productions) to parse any LL(1) grammar.
- If a grammar is changed, the parse table and list of productions will change, but the driver need not be changed.

Available on notes.arpanadhikari7.com.np

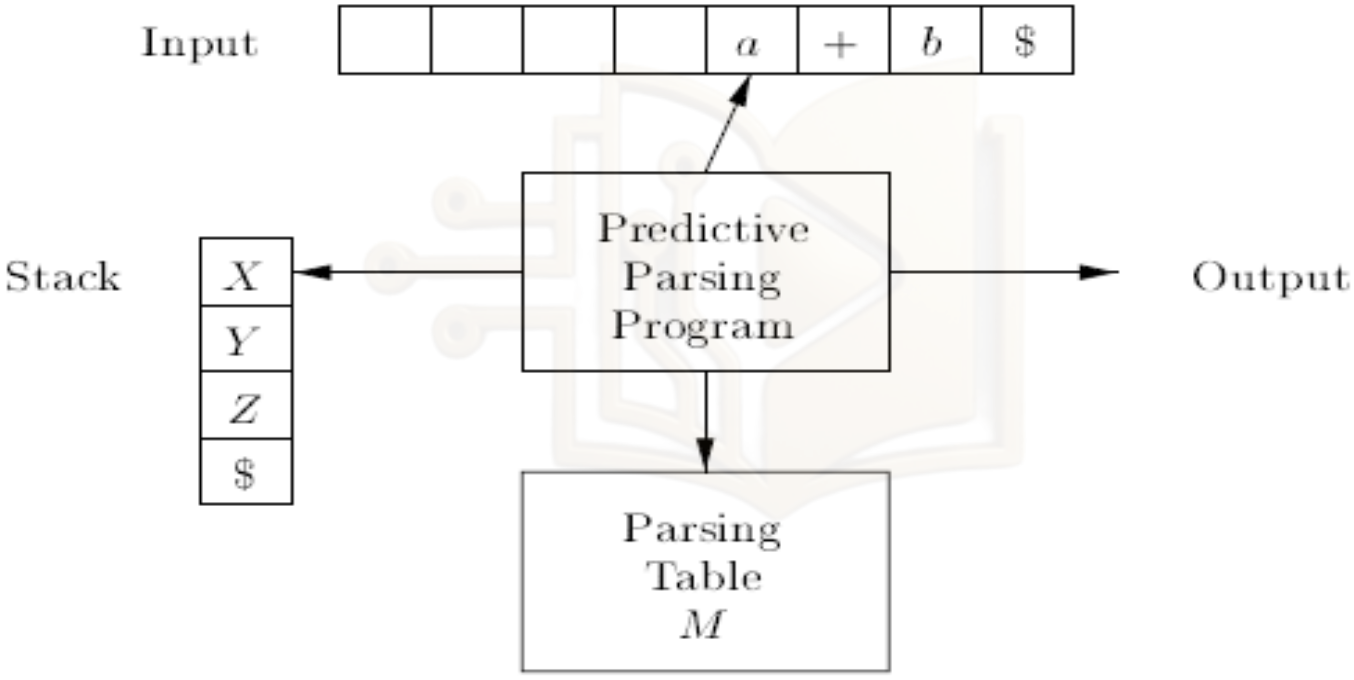


Figure Model of a table driven predictive parser

Available on notes.arpanadhikari7.com.np

let a be the first symbol of w;
let X be the top stack symbol;
while (X ≠ $) { /* stack is not empty */
if (X = a) pop the stack and let a be the next symbol of w;
else if (X is a terminal) error();
else if (M[X; a] is an error entry) error();
else if (M[X; a] = X → $Y_1\ Y_2 \ldots Y_k$) {
output the production X → $Y_1 Y_2 \ldots Y_k$;
pop the stack;
push Y_k. Y_{k-1}, … Y_1 onto the stack, with Y1 on top;
g
let X be the top stack symbol;
}

Figure: Predictive parsing algorithm

Available on notes.arpanadhikari7.com.np

Examples

1. Consider the grammar G given by:

$S \rightarrow aAa \mid BAa \mid \varepsilon$

$A \rightarrow cA \mid bA \mid \varepsilon$

$B \rightarrow b$

- Parse the word parsing of the word w = bcba

2. Given a grammar,

- $S \rightarrow aBa$
- $B \rightarrow bB \mid \varepsilon$
- Input: abba

Available on notes.arpanadhikari7.com.np

- Construct LL(1) parsing table of following grammar

$E \rightarrow T\ E'$

$E' \rightarrow +\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow *\ F\ T' \mid \varepsilon$

$F \rightarrow (\ E\ ) \mid id$

- And parse the string w=id+id*id

Available on notes.arpanadhikari7.com.np

Bottom-up Parsing

- Bottom-up parsers build a parse tree starting from its leaves and working toward its root.
- The parser constructs a leaf node in the tree for each word returned by the scanner.
- These leaves form the lower fringe of the parse tree.
- To build a derivation, the parser adds layers of non terminals on top of the leaves in a structure dictated by both the grammar and the partially completed lower portion of the parse tree.

Available on notes.arpanadhikari7.com.np

- The process of replacing a substring by a non-terminal in bottom-up parsing is called reduction. It is a reverse process of production. E.g. let's take a production rule S → aA
- Here, if replacing aA by S then such a grammar is called reduction.
- Consider the grammar
- S → aABe
- A → Abc | b
- B → d
- Now, the sentence **abbcde can be reduced to S as follows**
- abbcde
- aAbcde (replacing b by using A → b)
- aAde (replacing Abc by using A → Abc)
- aABe (replacing d by using B → d)
- S this is the starting symbol of the grammar.

Available on notes.arpanadhikari7.com.np

- A substring that can be replaced by a non-terminal when it matches its right sentential form is called a handle.

- If the grammar is unambiguous, then every right-sentential form of the grammar has exactly one handle.

Available on notes.arpanadhikari7.com.np

The LR(1) Parsing Algorithm

- The critical step in a bottom-up parser, such as a table-driven lr(1) parser, is
- to find the next handle.
- Efficient handle finding is the key to efficient bottom up parsing.
- An lr(1) parser uses a handle-finding automaton, encoded into two tables, called Action and Goto.

Key Terms in LR(1) Parsing

- LR(1) Items
- An LR(1) item is of the form:
- $[A \rightarrow \alpha \cdot \beta, a]$
  - $A \rightarrow \alpha\beta$ is a production rule.
  - The dot (·) indicates the current parsing position.
  - *a* is a **look ahead symbol** (terminal) that guides reductions.

Available on notes.arpanadhikari7.com.np

- **Closure Operation**
  - Expands an item set by including all possible productions that can follow the current non-terminal, along with their look aheads.
- **Goto Operation**
  - Computes the transition from a state when a symbol (terminal or non-terminal) is processed.
- **Canonical LR(1) Collection**
  - The set of all possible LR(1) item sets (states) constructed using **Closure** and **Goto**.

Available on notes.arpanadhikari7.com.np

Steps in LR(1) Parsing

- **Construct the LR(1) Item Sets**
  - Start with the initial item [S′→·S,$] (where S′ is a new start symbol and $ is the end marker).
  - Apply **Closure** and **Goto** to build all possible states.

- **Build the Parsing Table**
  - **Action Table**: Determines shifts/reduces for terminals.
  - **Goto Table**: Handles transitions for non-terminals.

- **Parsing Algorithm**
  - Uses a **stack** to keep track of states and symbols.
  - At each step, it consults the parsing table to decide whether to **shift**, **reduce**, **accept**, or **error**.

Available on notes.arpanadhikari7.com.np

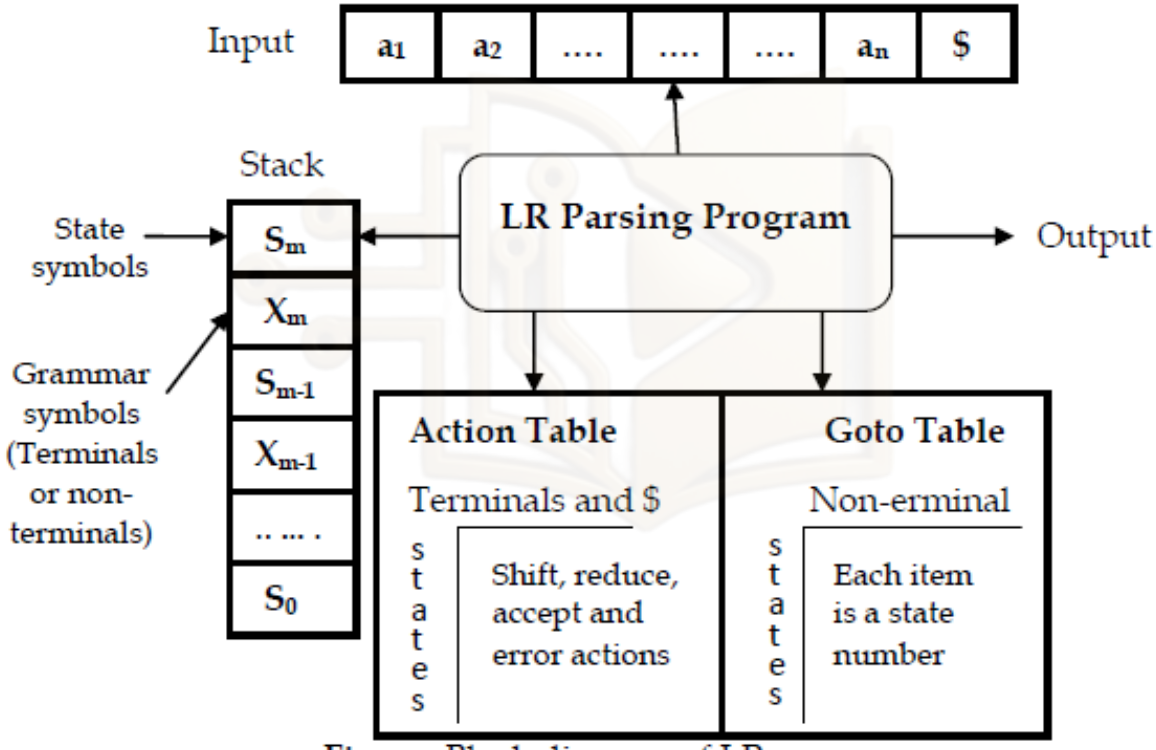


Figure: Block diagram of LR parser

Available on notes.arpanadhikari7.com.np

Building LR(1) Tables

- To construct Action and Goto tables, an lr(1) parser generator builds a model of the handle-recognizing automaton and uses that model to fill in the tables.
- The model, called the *canonical collection of sets of lr(1) items,* represents all of the possible states of the parser and the transitions between those states.
- Building LR(1) parsing tables involves several steps, from computing the LR(1) items and their closures to determining the goto and action entries.

Available on notes.arpanadhikari7.com.np

Steps involved are

- **Augment the Grammar**
- Start by augmenting the grammar with a new start symbol S′ and a production S′→S, where S is the original start symbol. This ensures the parser has a unique start state.
- Example:
- Original grammar:
  S→A
  A→aAb
  A→c
- Augmented grammar:
  S′→S
  S→A
  A→aAb
  A→c

Available on notes.arpanadhikari7.com.np

Compute LR(1) Items

- An **LR(1) item** is of the form [A→α·β,a], where:
- A→αβ is a production,
- The dot (··) marks the current position in parsing,
- a is a lookahead symbol (terminal or $).

Available on notes.arpanadhikari7.com.np

- **Closure of LR(1) Items**
- Given an item [A→α·Bβ,a], the closure includes:
- All items [B→·γ,b], where B→γ is a production, and b ∈ FIRST(βa).
- Example
- For [S′→·S,$], compute closure:
- Add [S→·A,$] (since FIRST(S$)=$FIRST(*S*$)=$).
- Then, for [S→·A,$], add [A→·aAb,$] and [A→·c,$].

Available on notes.arpanadhikari7.com.np

Construct the Canonical LR(1) Collection

- The canonical collection is the set of all possible LR(1) item sets (states) reachable from the initial state.
- **Goto Operation**
- Given a state I and a symbol X, compute:
- GOTO(I,X)=CLOSURE({[A→αX·β,a] | [$A \to a \cdot X\beta$, $a \in$ I})
- Repeat until no new states are generated.

Available on notes.arpanadhikari7.com.np

Example

- From I_0={[S′→·S,$],[S→·A,$],[A→·aAb,$],[A→·c,$]}
- GOTO(I_0,S)=I_1={[S′→S·,$]}
- GOTO(I_0,A)=I_2={[S→A·,$]}
- GOTO(I_0,a)=I_3={[A→a·Ab,$],[A→·aAb,b],[A→·c,b]}
- GOTO(I_0,c)=I_4={[A→c·,$]}
- Continue until all states are explored.

Available on notes.arpanadhikari7.com.np

Build the LR(1) Parsing Table

- The parsing table has two parts:
- **ACTION**[state, terminal]: Shift, Reduce, Accept, or Error.
- **GOTO**[state, non-terminal]: Next state after reducing.

Available on notes.arpanadhikari7.com.np

Algorithm for Construction of canonical-LR parsing tables.

1. Construct C′ = {I_0, I_1, …. , I_n}, the collection of sets of LR(1) items for G′.

2. State i of the parser is constructed from Ii. The parsing action for state i is determined as follows.

a. If [A→α·aβ,b] is in Ii and GOTO (Ii , a)=Ij, then set ACTION[i , a] to shift j . Here a must be a terminal.

b. If [A→α·,a] is in Ii, A ≠ S′, then set ACTION[i , a] to reduce A→α

c. If [S′ → S. , $] is in Ii, then set ACTION[i , $] to accept.“

- If any conflicting actions result from the above rules, we say the grammar is not LR(1). The algorithm fails to produce a parser in this case.

Available on notes.arpanadhikari7.com.np

3. The goto transitions for state i are constructed for all nonterminals A using the rule: If GOTO(Ii , A) = Ij , then GOTO[i , A] = j.

4. All entries not defined by rules (2) and (3) are made error.

5. The initial state of the parser is the one constructed from the set of items containing If [S′ → S. , $].

Available on notes.arpanadhikari7.com.np

Examples

Construct LR(1) parsing table of following grammar,

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$

Construct LR(1) parsing table of the following grammar

$S \rightarrow AaAb$

$S \rightarrow BbBa$

$A \rightarrow \in$

$B \rightarrow \in$

Available on notes.arpanadhikari7.com.np

Construct LR(1) parsing table of the following grammar

S->Aa | bAc | Bc | bBa

A->d

B->d

Construct LR(1) parsing table of the following grammar

S->CC

C->aC | d

Available on notes.arpanadhikari7.com.np